

```
require('dotenv').config();
const express =
require('express'), cors =
require('cors'), crypto =
require('crypto');
const bcrypt =
require('bcryptjs'), jwt =
require('jsonwebtoken');
const { Pool } = require('pg'), fs
= require('fs');

const { DATABASE_URL, JWT_SECRET,
FEDAPAY_SECRET_KEY,
FEDAPAY_WEBHOOK_SECRET,
FEDAPAY_API, PUBLIC_URL } =
process.env;
if (!DATABASE_URL) {
console.error('ERREUR : la
variable DATABASE_URL est
manquante.');
```

```
process.exit(1); }
if (!JWT_SECRET) {
console.error('ERREUR : la
variable JWT_SECRET est
manquante.');
```

```
process.exit(1); }
```

```
const pool = new Pool({
  connectionString: DATABASE_URL,
  ssl:
    /localhost|127\.\0\.\0\.\1/.test(DATABASE_URL) ? false : {
      rejectUnauthorized: false } });
const q = (text, params) =>
  pool.query(text, params);
const h = fn => (req, res, next)
  => fn(req, res).catch(next);
const normPhone = p => String(p ||
  '').replace(/\D/g, '');
const isId = v =>
  Number.isInteger(Number(v)) &&
  Number(v) > 0;
const paidSum = `COALESCE((SELECT
  SUM(amount) FROM payments WHERE
  student_id=s.id AND
  status='paid'),0)::int`;

const app = express();
app.use(cors());

// ---- Webhook Fedapay (corps
brut, AVANT express.json) ----
app.post('/api/webhooks/fedapay',
```

```

express.raw({ type: '*/*' }),
h(async (req, res) => {
    const sig = req.get('x-fedapay-
signature') || '';
    const t = (sig.match(/t=(\d+)/)
|| [])[1], s = (sig.match(/s=([a-
f0-9]+)/) || [])[1];
    const expected =
crypto.createHmac('sha256',
FEDAPAY_WEBHOOK_SECRET ||
'').update(`${t}.${req.body}`).dig
est('hex');
    if (!s || s.length !==
expected.length ||
!crypto.timingSafeEqual(Buffer.fro
m(s), Buffer.from(expected)))
        return res.sendStatus(400);
    const ev = JSON.parse(req.body),
tx = ev.entity || {};
    const status = ev.name ===
'transaction.approved' ? 'paid' :
ev.name === 'transaction.declined'
? 'failed' : null;
    if (status) await q('UPDATE
payments SET status=$1 WHERE
provider_ref=$2', [status,
String(tx.id)]);

```

```
    res.sendStatus(200);
  }));

app.use(express.json());

// ---- Auth ----
const sign = u => jwt.sign({ uid:
u.id, sid: u.school_id, role:
u.role }, JWT_SECRET, { expiresIn:
'7d' });
const auth = (req, res, next) => {
  try { req.user =
jwt.verify((req.get('authorization')
|| '').replace('Bearer ', ''),
JWT_SECRET); next(); }
  catch { res.status(401).json({
error: 'Non autorisé' }); }
};

app.post('/api/auth/register',
h(async (req, res) => {
  const { school, email, password
} = req.body;
  if (!school || !email ||
!password || password.length < 8)
return res.status(400).json({
error: 'Champs invalides (mot de
```

```

    passe 8+ car.)' });
    if ((await q('SELECT 1 FROM
users WHERE email=$1',
[email])).rowCount) return
res.status(409).json({ error:
'Email déjà utilisé' });
    const sid = (await q('INSERT
INTO schools(name) VALUES($1)
RETURNING id',
[school])).rows[0].id;
    const uid = (await q('INSERT
INTO
users(school_id,email,password_has
h) VALUES($1,$2,$3) RETURNING id',
[sid, email,
bcrypt.hashSync(password,
10)]))).rows[0].id;
    res.json({ token: sign({ id:
uid, school_id: sid, role:
'director' }) });
}));

app.post('/api/auth/login',
h(async (req, res) => {
    const u = (await q('SELECT *
FROM users WHERE email=$1',
[req.body.email])).rows[0];

```

```

    if (!u ||
    !bcrypt.compareSync(req.body.password || '', u.password_hash))
    return res.status(401).json({
    error: 'Identifiants incorrects'
    });
    res.json({ token: sign(u) });
  }));

// ---- Élèves (directeur) ----
app.get('/api/students', auth,
h(async (req, res) => {
  res.json((await q(`SELECT s.*,
  ${paidSum} AS paid FROM students s
  WHERE school_id=$1 ORDER BY name`,
  [req.user.sid]))).rows);
}));

app.post('/api/students', auth,
h(async (req, res) => {
  const { name, class: cls,
  parent_phone, total_fee } =
  req.body;
  if (!name || !parent_phone || !
  (total_fee >= 0)) return
  res.status(400).json({ error:
  'Champs invalides' });

```

```

    const r = await q('INSERT INTO
students(school_id,name,class,pare
nt_phone,total_fee)
VALUES($1,$2,$3,$4,$5) RETURNING
id',
    [req.user.sid, name, cls ||
'', normPhone(parent_phone),
Math.round(total_fee)]);
    res.status(201).json({ id:
r.rows[0].id });
  }));

app.delete('/api/students/:id',
auth, h(async (req, res) => {
  if (!isId(req.params.id)) return
res.status(400).json({ error:
'Identifiant invalide' });
  await q('DELETE FROM students
WHERE id=$1 AND school_id=$2',
[req.params.id, req.user.sid]);
  res.sendStatus(204);
}));

// ---- Paiements manuels ----
app.post('/api/payments', auth,
h(async (req, res) => {
  const { student_id, amount,

```

```

method } = req.body;
    if (!isId(student_id)) return
res.status(400).json({ error:
'Élève invalide' });
    const s = (await q(`SELECT
s.total_fee, ${paidSum} AS paid
FROM students s WHERE id=$1 AND
school_id=$2`, [student_id,
req.user.sid])).rows[0];
    if (!s) return
res.status(404).json({ error:
'Élève introuvable' });
    if (!(amount > 0) || amount >
s.total_fee - s.paid) return
res.status(400).json({ error:
'Montant invalide' });
    const r = await q('INSERT INTO
payments(school_id,student_id,amou
nt,method) VALUES($1,$2,$3,$4)
RETURNING id',
    [req.user.sid, student_id,
Math.round(amount), method ||
'Espèces']);
    res.status(201).json({ id:
r.rows[0].id, remaining:
s.total_fee - s.paid - amount });
}));

```

```

app.get('/api/dashboard', auth,
h(async (req, res) => {
  const due = (await q('SELECT
COALESCE(SUM(total_fee),0)::int AS
v FROM students WHERE
school_id=$1',
[req.user.sid])).rows[0].v;
  const paid = (await q("SELECT
COALESCE(SUM(amount),0)::int AS v
FROM payments WHERE school_id=$1
AND status='paid'",
[req.user.sid])).rows[0].v;
  res.json({ due, paid, remaining:
due - paid, rate: due ?
Math.round(paid / due * 100) : 0
});
}));

// ---- Espace parent (par
téléphone) ----
// TODO production : ajouter un
code OTP par SMS avant d'afficher
ces données.
app.get('/api/parent/students',
h(async (req, res) => {
  res.json((await q(`SELECT s.id,

```

```
s.name, s.class, s.total_fee,
${paidSum} AS paid FROM students s
WHERE parent_phone=$1`,

[normPhone(req.query.phone)])))).row
s);
}));

app.post('/api/parent/pay',
h(async (req, res) => {
  const { student_id, amount,
phone } = req.body;
  if (!isId(student_id)) return
res.status(400).json({ error:
'Demande invalide' });
  const s = (await q(`SELECT s.*,
${paidSum} AS paid FROM students s
WHERE id=$1 AND parent_phone=$2`,
[student_id,
normPhone(phone)])))).rows[0];
  if (!s || !(amount > 0) ||
amount > s.total_fee - s.paid)
return res.status(400).json({
error: 'Demande invalide' });
  try {
    const H = { 'Content-Type':
'application/json', Authorization:
```

```

`Bearer ${FEDAPAY_SECRET_KEY}` };
    const tx = await (await
fetch(`${FEDAPAY_API}/transactions
`, { method: 'POST', headers: H,
body: JSON.stringify({
    description: `Scolarité -
${s.name}`, amount:
Math.round(amount), currency: {
iso: 'XOF' },
    callback_url:
`${PUBLIC_URL}/api/webhooks/fedapa
y` }) })).json();
    const id =
(tx['v1/transaction'] || tx).id;
    const tk = await (await
fetch(`${FEDAPAY_API}/transactions
/${id}/token`, { method: 'POST',
headers: H })).json();
    await q("INSERT INTO
payments(school_id,student_id,amou
nt,method,status,provider_ref)
VALUES($1,$2,$3,'Fedapay','pending
',$4)",
    [s.school_id, s.id,
Math.round(amount), String(id)]);
    res.json({ payment_url: tk.url
});

```

```

    } catch (e) {
res.status(502).json({ error:
'Passerelle de paiement
indisponible' }); }
}));

// ---- Pages et erreurs ----
const pub =
fs.existsSync(__dirname +
'/public/index.html') ? __dirname
+ '/public/index.html' : __dirname
+ '/index.html';
app.get('/', (req, res) =>
res.sendFile(pub));
app.use(express.static(__dirname +
'/public'));
app.use((err, req, res, next) => {
  console.error(err);
  res.status(err.code === '23505'
? 409 : 500).json({ error:
err.code === '23505' ? 'Déjà
existant' : 'Erreur serveur' });
});

(async () => {
  await
q(fs.readFileSync(__dirname +

```

```
'/schema.sql', 'utf8'));  
  app.listen(process.env.PORT ||  
3000, () => console.log('EduCompta  
API prête (PostgreSQL)'));  
})();catch(e => {  
  console.error('Connexion à la base  
impossible :', e.message);  
  process.exit(1); });
```